JAIN DEEMED-TO-BE UNIVERSITY | FACULTY OF ENGINEERING AND TECHNOLOGY

PROGRAMME: B. TECH COMPUTER SCINCE AND ENGINEERING

MINI PROJECT REPORT

Python Programming

(3rd Semester)

Student Name & USN :	**Akshath Maurya (25BTRPW010)** **Divyanshu Nahata (25BTRPW035)** **Ekansh Kumar (25BTRPW037)** **Daniel Christopher (25BTRPW030)** **J S Rahul (25BTRPW048)** **Ardra Rajesh (25BTRPW016)** **Gurram Pati Harsha Vardhan Rathod (25BTRPW041)**	**Subject Name:**	**Python Programming**
		Faculty Name:	**Ms.Sujithra. G.**
Class/Section:	**CSE – PwC A**	**Date of Demo:**	**13 – 08 - 2026**
Project uploaded GitHub link	**https://ardra07-rajesh.github.io/student_grade_evaluation.py/**	**Date of Report Submission:**	**13 – 08 – 2026**
Website link	**marks-and-regret.html**		

STUDENT GRADE EVALUATION & PERSONALIZED FEEDBACK SYSTEM

1. Problem Statement

The aim of this project is to design and develop a **Student Grade Evaluation & Personalized Feedback System** using Python that can automatically evaluate a student's academic performance based on the marks entered.

Students usually receive their marks after an examination, but understanding the marks may require additional calculations. They may need to determine their grade, result, grade point, performance level, and the number of marks required to reach the next grade. Performing these calculations manually can take additional time and may lead to calculation errors.

The application solves this problem by accepting the student's name and marks and automatically generating a complete performance report. The program determines the student's grade, pass or fail result, grade point, and performance status based on predefined grading rules.

The system also provides marks based personalized feedback and calculates the number of additional marks required to reach the next grade. Input validation is included to handle empty names, nonnumeric marks, negative values, and marks above 100.

The project is designed to make academic evaluation faster, organized, and easier to understand while providing the student with useful information about their current performance and next target.

2. Project Description

Student Grade Evaluation & Personalized Feedback System is a Python based console application developed to evaluate and interpret student marks.

The program starts by asking the user to enter the student's name and marks. The entered name is validated to ensure that it is not empty. The marks are converted into a floating point value and checked to ensure that they fall within the valid range of 0 to 100.

After successful validation, the program evaluates the marks using predefined grading conditions. The system assigns a grade from A+ to F and determines the corresponding grade point and result.

The program also assigns a performance status based on the marks. The status ranges from **TOPPER MODE** for high scores to **ACADEMIC EMERGENCY** for failing marks.

A separate feedback function generates a marks based message for the student. This gives the student a simple interpretation of the result instead of displaying only the numerical score.

Another feature of the program calculates the marks required to reach the next grade. For example, if a student scores 78 marks, the system identifies the grade as B and informs the student that 2 more marks are required to reach grade A.

Finally, the program displays all the calculated information in a structured result report containing the student's name, marks, grade, result, grade point, performance status, personalized feedback, and next grade target.

The application uses separate functions for different operations. This modular structure makes the program easier to understand, test, debug, and maintain.

Grading System

Marks	Grade	Grade Point	Result
90 to 100	A+	10	Pass
80 to 89.99	A	9	Pass
70 to 79.99	B	8	Pass
60 to 69.99	C	7	Pass
50 to 59.99	D	6	Pass
0 to 49.99	F	0	Fail

3. Concepts Used in the Program

The following Python programming concepts are used in this project:

1. Functions

Functions are used to divide the program into smaller and organized sections.
The main functions used in the project are:

- calculate_grade_and_result()
- get_feedback()
- marks_needed_for_next_grade()
- display_report()
- main()

Each function performs a specific task. This makes the program easier to understand, test, debug, and modify.

2. Conditional Statements

if, elif, and else statements are used to evaluate the marks and make decisions.
They are used for:

- Assigning grades
- Determining pass or fail status
- Assigning grade points
- Generating performance status
- Generating personalized feedback
- Calculating the next grade target

For example, the program checks whether the marks are greater than or equal to 90 before assigning grade A+.

3. Variables and Data Types

Different data types are used throughout the program.

- Strings are used for student names, grades, results, status, and feedback.
- Integers are used for grade points.
- Floating point values are used for student marks and next grade calculations.
- Boolean expressions are used during validation and conditional checks.

4. User Input

The input() function is used to collect information from the user.

The program collects:

- Student name
- Student marks

For example:

name = input("Enter student name: ").strip()

The .strip() function removes unnecessary spaces from the beginning and end of the entered name.

5. Type Conversion

The marks entered by the user are initially received as a string. The program converts the input into a floating point value using the float() function.

marks = float(input("Enter marks (0-100): "))

This allows the program to accept decimal marks such as 78.5.

6. Exception Handling

The try-except block is used to handle invalid numerical input.

If the user enters text instead of a numerical value, the program catches the ValueError and displays an appropriate error message instead of terminating unexpectedly.

```
try:
    marks = float(input("Enter marks (0-100): "))
except ValueError:
    print("Error: Invalid input.")
```

7. Data Validation

The program validates the information entered by the user.

It checks whether:

- The student name is not empty.
- The entered marks are numerical.
- The marks fall between 0 and 100.

For example:

```
if not (0 <= marks <= 100):
    print("Error: Marks must be between 0 and 100.")
```

This prevents invalid marks from being processed.

8. Return Values

Functions return calculated values to other parts of the program.

The calculate_grade_and_result() function returns four values:

- Grade
- Result
- Grade Point
- Performance Status

For example:

return "A+", "Pass", 10, "TOPPER MODE"

The returned values are then used by the main() and display_report() functions.

9. String Formatting
F strings are used to display the student information in a structured format.
For example:
print(f" Marks : {marks:.2f}%")
print(f" Grade : {grade}")
The formatting makes the final result easier to read.

10. Arithmetic Operations
Arithmetic operations are used to calculate the marks required for reaching the next grade.
For example, if a student obtains 78 marks, the program calculates:
80 - 78 = 2
Therefore, the student needs 2 more marks to reach grade A.

11. Modular Programming
The program follows a modular structure where different functions handle different tasks.
The basic flow of the application is:
User Input → Validation → Grade Calculation → Feedback Generation → Next Grade Calculation → Final Report
This structure improves code organization and makes individual functions easier to test.

12. Main Function
The main() function controls the overall execution of the program.
It:

- Displays the welcome message.
- Accepts the student name.
- Validates the name.
- Accepts the marks.
- Validates the marks.
- Calls the grade calculation function.
- Displays the final report.

The program starts execution using:
if __name__ == "__main__":
main()

4.Program Code

```
def calculate_grade_and_result(marks):
    if marks >= 90:
        return "A+", "Pass, TRY TO GET 1st NEXT TIME :(", 10, "TOPPER MODE"
    elif marks >= 80:
        return "A", "Pass, ONLY 80? NO 90 :(", 9, "ALMOST TOPPER"
    elif marks >= 70:
        return "B", "Pass, OK BUT LOOK AT OTHER GOOD STUDYING KIDS AND LEARN :(", 8, "NOT
BAD"
```

```
    elif marks >= 60:
        return "C", "Pass, HUGE DISAPPOINTMENT TO FAMILY :(", 7, "YOU CAN DO BETTER"
    elif marks >= 50:
        return "D", "Pass, JUST PASS :(", 6, "BARELY SURVIVING"
    else:
        return "F", "Fail, BRO WE NEED A COMEBACK", 0, "ACADEMIC EMERGENCY"



def get_feedback(marks):
    if marks >= 90:
        return "Bro is cooking! Keep the topper energy alive."
    elif marks >= 80:
        return "So close to 90. A little more grinding next time."
    elif marks >= 70:
        return "Decent score. Time to stop procrastinating."
    elif marks >= 60:
        return "You passed, but your books are asking for attention."
    elif marks >= 50:
        return "Congratulations! You escaped the danger zone."
    else:
        return "It's not over. Wake up, study harder and comeback."



def marks_needed_for_next_grade(marks):
    if marks >= 90:
        return "MAXIMUM GRADE ACHIEVED!"
    elif marks >= 80:
        return f"{90 - marks:.2f} more marks needed for A+"
    elif marks >= 70:
        return f"{80 - marks:.2f} more marks needed for A"
    elif marks >= 60:
        return f"{70 - marks:.2f} more marks needed for B"
    elif marks >= 50:
        return f"{60 - marks:.2f} more marks needed for C"
    else:
        return f"{50 - marks:.2f} more marks needed to PASS"
```

```
def display_report(name, marks, grade, result, grade_point, status):
    print("\n" + "=" * 45)
    print("        STUDENT RESULT & REALITY CHECK")
    print("=" * 45)

    print(f" Name          : {name}")
    print(f" Marks         : {marks:.2f}%")
    print(f" Grade         : {grade}")
    print(f" Result        : {result}")
    print(f" Grade Point   : {grade_point}/10")
    print(f" Status        : {status}")

    print("-" * 45)

    # Personalized feedback
    feedback = get_feedback(marks)
    print(f" Reality Check : {feedback}")

    # Next grade information
    next_grade = marks_needed_for_next_grade(marks)
    print(f" Next Target   : {next_grade}")

    print("=" * 45)



def main():
    print("\n" + "*" * 45)
    print("       WELCOME TO THE MARKS & REGRET")
    print("                  DEPARTMENT")
    print("*" * 45)

    name = input("\nEnter student name: ").strip()
```

```
    # Validate name
    if not name:
        print("Error: Name cannot be empty.")
        return

    # Validate marks
    try:
        marks = float(input("Enter marks (0-100): "))

        if not (0 <= marks <= 100):
            print("\nError: Marks must be between 0 and 100.")
            print("Bro entered illegal marks 💀")
            return

    except ValueError:
        print("\nError: Invalid input.")
        print("Please enter numbers only. Python is not a mind reader 😭")
        return

    # Calculate result
    grade, result, grade_point, status = calculate_grade_and_result(marks)

    # Display report
    display_report(
        name,
        marks,
        grade,
        result,
        grade_point,
        status
    )



# Start the program
if __name__ == "__main__":
    main()
```

5.Output Screenshots

```
Enter marks (0-100): 90

==============================================
        STUDENT RESULT & REALITY CHECK
==============================================
 Name           : XY
 Marks          : 90.00%
 Grade          : A+
 Result         : Pass, TRY TO GET 1st NEXT TIME :(
 Grade Point    : 10/10
 Status         : TOPPER MODE
----------------------------------------------
 Reality Check : Bro is cooking! Keep the topper energy alive.
 Next Target   : MAXIMUM GRADE ACHIEVED!
==============================================
PS C:\Users\Asus\Documents>
```

Figure 1: Output for marks = 90 (Grade A+, Topper Mode)

```
==============================================
 Name           : XYZ
 Marks          : 68.00%
 Grade          : C
 Result         : Pass, HUGE DISAPPOINTMENT TO FAMILY :(
 Grade Point    : 7/10
 Status         : YOU CAN DO BETTER
----------------------------------------------
 Reality Check : You passed, but your books are asking for attention.
 Next Target   : 2.00 more marks needed for B
==============================================
PS C:\Users\Asus\Documents>
```

Figure 2: Output for marks = 68 (Grade C, You Can Do Better)

6. Conclusion

The **Student Grade Evaluation & Personalized Feedback System** successfully demonstrates how Python can be used to develop a simple and practical academic evaluation application.

The project accepts student information and marks, validates the entered data, calculates the appropriate grade and grade point, determines the pass or fail result, and generates a performance status based on the student's marks.

The application also provides marks based personalized feedback and calculates the number of marks required to reach the next grade. These features help students understand their current performance and identify a clear target for improvement.

Through this project, the team practically implemented important Python programming concepts such as functions, conditional statements, variables, data types, user input, type conversion, exception handling, data validation, arithmetic operations, string formatting, return values, and modular programming.

The modular structure of the program makes the code easier to understand, test, debug, and maintain. Testing with valid inputs, boundary values, and invalid inputs confirmed that the implemented grading and validation rules work correctly.

The current application processes one student at a time and displays the result through the console. It can be further developed by adding multiple student processing, CSV or database storage, student performance history, graphical reports, analytics, and AI based study recommendations.

Overall, this project provided the team with practical experience in Python programming and demonstrated how programming concepts can be combined to create a useful academic evaluation system.